

Person 2 -- Human-Gate, Consent, and Audit Trail

NHAA Trauma Triage System -- Standalone Build Specification

How to Use This Document

This is a complete, standalone build specification. Build every phase directly, in order, without waiting for or requiring live coordination with any other team member. Wherever your code needs to receive data from another person's component, this document tells you exactly what mock/fake version of that data to build for your own testing. Your real component, once finished, will accept the same shape from the real component later at merge time with no changes needed, as long as you follow the shared contract exactly.

Your Role

You own the system's entire safety and ethics architecture: the rule that police, witness protection, and emergency escalation can never fire without a second human's explicit approval, enforced as code, not as a suggestion. This is the single most important correctness guarantee in the whole project -- test it more aggressively than anything else you build.

The Shared Contract

You consume a `risk_vector` and `svi` object shaped like this (produced by Person 1, but you will build a fake version of this for your own testing in Phase 1):

```
{ "scores": { "acute_distress": 0.0, "self_harm_risk": 0.0,
"fear_of_retaliation": 0.0 }, "bucket": "critical" }
```

You emit this shape (consumed by Person 3's interface, but they are building against a mock of this too, so you do not need to coordinate):

```
{
  "type": "action_update",
  "action": {
    "action": "counseling_referral | legal_aid | police_intervention
| witness_protection | emergency_escalation",
    "state": "suggested | agent_confirmed | awaiting_senior |
executed | rejected | overridden",
    "requires_senior": true,
    "agent_id": "string", "senior_id": "string or null",
    "override_reason": "string or null"
  }
}
```

Phase 1 -- Build a Fake Risk-Vector Generator for Your Own Testing

Since you do not need Person 1's real pipeline to build and test your own module, write a small Python function `fake_risk_vector(bucket: str) -> dict` that returns hand-crafted score dictionaries for each of the four buckets (Low, Moderate, High, Critical), matching the shared contract shape above. Use this throughout your entire build to drive your own tests -- you will swap in the real pipeline output only at the very end, at merge time.

Phase 2 -- Build the Three Core Modules, Tested Alone

decision_aid.py -- Define an `ActionType` enum with five values (`counseling_referral`, `legal_aid`, `police_intervention`, `witness_protection`, `emergency_escalation`). Define an `ActionState` enum with six values (`suggested`, `agent_confirmed`, `awaiting_senior`, `executed`, `rejected`, `overridden`). Define a Python set called `REQUIRES_SENIOR_SIGNOFF` containing exactly `{police_intervention, witness_protection, emergency_escalation}`. Build a `DecisionAidLayer` class with these methods: - `suggest_for_bucket(call_id, bucket)` -- returns the correct list of actions for that bucket (Low: none, Moderate: counseling only, High: counseling + legal_aid, Critical: all five). - `agent_confirm(call_id, action, agent_id, caller_consent_given)` -- if the action is in `REQUIRES_SENIOR_SIGNOFF`, move it to `awaiting_senior`. Otherwise, if `caller_consent_given` is true, move it directly to `executed`. - `senior_approve(call_id, action, senior_id)` -- **must raise a ValueError if the action's current state is not exactly awaiting_senior**. This is the single most important line of code in your entire build. Write this check first, before anything else in this method. - `senior_reject(call_id, action, senior_id, reason)` and `agent_override(call_id, action, agent_id, reason)` -- straightforward state transitions to `rejected` / `overridden`.

Write a test script that: (1) confirms a Critical-bucket call correctly suggests all five actions, (2) confirms calling `agent_confirm` on `police_intervention` moves it to `awaiting_senior` and NOT `executed`, (3) confirms calling `senior_approve` on an action that is NOT in `awaiting_senior` state raises an error, (4) confirms calling `senior_approve` correctly on an action that IS in `awaiting_senior` moves it to `executed`.

consent.py -- Define a `ConsentPurpose` enum with three values: `recording`, `ai_analysis`, `data_for_improvement`. Build a `ConsentState` class with `grant(purpose)`, `decline(purpose)`, `revoke(purpose)` methods, each updating an internal dictionary, and a `revoked` property that returns True if and only if `ai_analysis` is currently not granted. Test: `grant ai_analysis`, `confirm revoked` is False; call `revoke(ai_analysis)`, `confirm revoked` immediately becomes True.

audit_log.py -- Use Python's built-in `sqlite3` module. Create a table `audit_events` with columns `id`, `call_id`, `event_type`, `payload_json`, `created_at`, `prev_hash`, `this_hash`. Build a `log(event_type, call_id, payload)` method that: serializes the payload to JSON, computes `this_hash = sha256(prev_hash + event_type + payload_json +`

created_at) using Python's built-in hashlib, fetches the most recent row's this_hash to use as this new row's prev_hash (use "0"*64 as the genesis prev_hash for the very first row), and inserts the row. Build a verify_chain(call_id) method that walks every row for that call in order and confirms each row's prev_hash matches the previous row's this_hash, returning False the moment a mismatch is found. Test: write 5 events, confirm verify_chain returns True; then directly edit one row's payload_json in the SQLite file using a separate script, and confirm verify_chain now returns False.

Phase 2b -- Confidence Gating and the Two Missing Safety-Net Layers

The master solution document describes three safety-net layers (Section 2.9): the agent's real-time override (already covered by agent_override above), scheduled follow-up check-ins, and supervisor review. Only the first is built so far. Build the other two now, as part of this same phase, before moving on.

Confidence gating (confidence_gate.py). Build a function should_force_human_review(overall_confidence: float, bucket: str) -> bool that returns True if overall_confidence (a value between 0 and 1, produced by Person 1's pipeline as overall_confidence in their output) is below 0.5, regardless of what bucket the SVI computed. When this returns True, the calling code must treat the case as requiring mandatory human review before any suggested action is shown as actionable, even if the computed bucket would normally allow faster action. Test: confirm a High-bucket case with overall_confidence = 0.3 is forced into mandatory-review treatment rather than proceeding normally.

Scheduled safety check-ins (safety_checkins.py). Build a SQLite table safety_checkins with columns case_id, risk_bucket_at_call, scheduled_72h_date, scheduled_7d_date, completed_72h, completed_7d, escalated_no_response. Build a function schedule_safety_checkins(case_id, bucket, call_timestamp) that, only if bucket is low or moderate, inserts a row with the 72-hour and 7-day dates calculated from call_timestamp. Build a function mark_checkin_complete(case_id, which: str) and a function get_overdue_checkins(current_time) that returns every row where a scheduled date has passed and the corresponding completed flag is still False -- this is the human-callable list of people who need a human-initiated follow-up call. Test: schedule a checkin with a call_timestamp in the past such that the 72-hour mark has already passed, confirm it appears in get_overdue_checkins.

Supervisor spot-check queue (supervisor_queue.py). Build a function flag_for_supervisor_review(case_id, bucket) -> bool that returns True always if bucket == "critical" (100% of Critical cases reviewed), and returns True for Low/Moderate cases with a 5% random chance (random.random() < 0.05) otherwise. Build a function get_review_queue() returning every case_id flagged this way that hasn't yet been marked reviewed, plus a mark_reviewed(case_id, supervisor_id, notes) function. Test: call flag_for_supervisor_review 1000 times with bucket="low" and confirm roughly 5% (between 30 and 70) return True; confirm every single call with bucket="critical" returns True.

Override-rate tracking (drift_monitor.py). Since your `decision_aid.py` already records `agent_override` events, build a small function `compute_override_rate(events: list, language: str, window_hours: int = 24) -> float` that counts what fraction of actions for a given language within the time window were overridden by the agent rather than confirmed as suggested. This is the proxy signal for the "silent accuracy drift" concern in Section 2.11 of the solution document -- document clearly in your code comments that this number is only meaningful once real call volume exists, and that a single test run will not produce a meaningful rate; the function itself, however, must be built and correct now.

Phase 3 -- Combine Into a Single Testable Flow

Write one script that simulates a full Critical-risk call end-to-end using your Phase 1 fake risk-vector generator: consent is granted, a Critical bucket is reached, all five actions are suggested, counseling is agent-confirmed and executed, police_intervention is agent-confirmed (moves to awaiting_senior), then senior-approved (moves to executed), and every step is written to the audit log. At the end, call `verify_chain` and confirm it returns `True`. This script is your proof that the whole safety architecture works correctly end to end, using only your own code.

Phase 4 -- Adversarial Testing (Try to Break Your Own Gate)

Write a test for each of these specific bypass attempts, and confirm every single one fails safely: 1. Call `senior_approve` twice in a row on the same action. 2. Call `senior_approve` on an action that was never suggested for that call at all. 3. Call `agent_confirm` twice on the same `police_intervention` action. 4. Attempt to directly set an action's state to `executed` by manipulating the object's internal state from outside the class methods (confirm your class design makes this awkward/unnatural to do, even if Python cannot fully prevent it -- document this as a known language-level limitation if so).

Phase 5 -- Package for Merge

Wrap your three modules behind one clear interface: a `SafetyGateModule` class exposing `process_bucket_change(call_id, bucket, risk_scores)`, `handle_agent_action(...)`, and `handle_senior_action(...)` methods, matching the shared contract's output shape exactly. Write a `README_safety_gate.md` explaining exactly how another module (Person 1's real pipeline, once merged in) should call your functions, and include the full output of your Phase 4 adversarial tests as proof the gate holds.